

customer_repeat_purchase_analysis

September 18, 2026

```
[5]: import os
os.listdir()

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.read_csv("customer_features_90d_202609171623.csv")

df.head()
```

```
[5]:  customer_id  first_purchase_date  acquisition_month  first_order_value  \
0         12346  2009-12-14 08:34:00          2009-12           45.00
1         12347  2010-10-31 14:20:00          2010-10          611.53
2         12348  2010-09-27 14:59:00          2010-09          222.16
3         12349  2010-04-29 13:20:00          2010-04         1068.52
4         12350  2011-02-02 16:01:00          2011-02          334.40

      first_order_items  first_order_products  first_order_country  \
0                   10                      1      United Kingdom
1                   509                      40              Iceland
2                   373                      20              Finland
3                   473                      46                Italy
4                   197                      17              Norway

      second_purchase_date  days_to_second_purchase  repeat_90d
0  2009-12-14 11:00:00           0.0                1
1  2010-12-07 14:57:00          37.0                1
2  2010-12-16 19:09:00          80.0                1
3  2010-05-18 09:57:00          19.0                1
4                NaN           NaN                0
```

```
[6]: print("Dataset shape:", df.shape)

df.info()
```

```
Dataset shape: (5281, 10)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5281 entries, 0 to 5280
```

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	customer_id	5281 non-null	int64
1	first_purchase_date	5281 non-null	object
2	acquisition_month	5281 non-null	object
3	first_order_value	5281 non-null	float64
4	first_order_items	5281 non-null	int64
5	first_order_products	5281 non-null	int64
6	first_order_country	5281 non-null	object
7	second_purchase_date	4023 non-null	object
8	days_to_second_purchase	4023 non-null	float64
9	repeat_90d	5281 non-null	int64

dtypes: float64(2), int64(4), object(4)

memory usage: 412.7+ KB

```
[7]: df.isnull().sum()
```

```
[7]: customer_id          0
first_purchase_date      0
acquisition_month        0
first_order_value        0
first_order_items        0
first_order_products     0
first_order_country      0
second_purchase_date    1258
days_to_second_purchase 1258
repeat_90d              0
dtype: int64
```

```
[8]: df["repeat_90d"].value_counts()
```

```
[8]: 0    2785
1    2496
Name: repeat_90d, dtype: int64
```

```
[9]: df["repeat_90d"].value_counts(normalize=True) * 100
```

```
[9]: 0    52.736224
1    47.263776
Name: repeat_90d, dtype: float64
```

```
[10]: df[["first_order_value",
        "first_order_items",
        "first_order_products"]].describe()
```

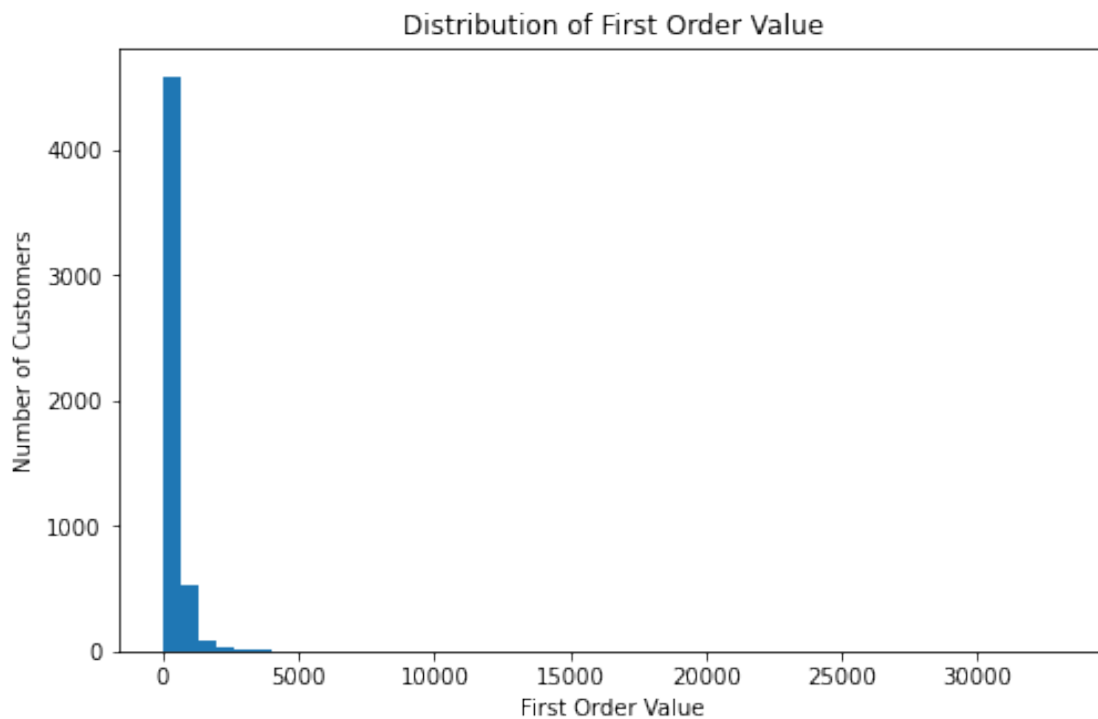
```
[10]:
```

	first_order_value	first_order_items	first_order_products
count	5281.000000	5281.000000	5281.000000
mean	416.472990	261.952471	23.721076
std	758.061712	1294.772583	22.456971
min	1.300000	1.000000	1.000000
25%	168.310000	80.000000	9.000000
50%	296.620000	153.000000	18.000000
75%	453.010000	281.000000	31.000000
max	33167.800000	87167.000000	250.000000

```
[11]: plt.figure(figsize=(8, 5))
plt.hist(df["first_order_value"],bins=50)

plt.xlabel("First Order Value")
plt.ylabel("Number of Customers")
plt.title("Distribution of First Order Value")

plt.show()
```

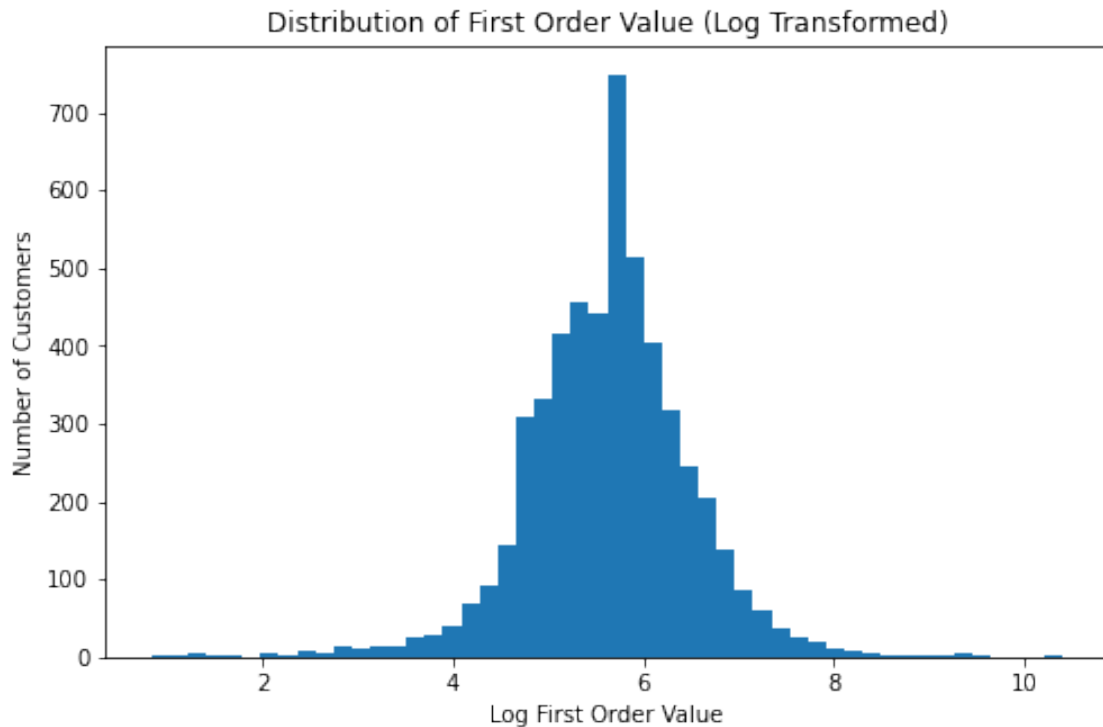


```
[12]: df["log_first_order_value"] = np.log1p(
        df["first_order_value"])

plt.figure(figsize=(8, 5))
plt.hist(df["log_first_order_value"],bins=50)
```

```
plt.xlabel("Log First Order Value")
plt.ylabel("Number of Customers")
plt.title("Distribution of First Order Value (Log Transformed)")

plt.show()
```



```
[13]: df[["first_order_value",
         "first_order_items",
         "first_order_products"]].skew()
```

```
[13]: first_order_value      21.113459
      first_order_items      58.413964
      first_order_products      2.691172
      dtype: float64
```

```
[17]: df["log_first_order_value"] = np.log1p(
        df["first_order_value"])
      df["log_first_order_items"] = np.log1p(
        df["first_order_items"])
      df["log_first_order_products"] = np.log1p(
        df["first_order_products"])
```

```
[18]: df[
      [
          "log_first_order_value",
          "log_first_order_items",
          "log_first_order_products"
      ]
      ].skew()
```

```
[18]: log_first_order_value    -0.297148
      log_first_order_items    -0.759286
      log_first_order_products  -0.434981
      dtype: float64
```

```
[20]: repeat_comparison = df.groupby("repeat_90d")[
      ["first_order_value",
       "first_order_items",
       "first_order_products"]].median()
      repeat_comparison
```

```
[20]:
```

	first_order_value	first_order_items	first_order_products
repeat_90d			
0	257.89	135.0	17.0
1	315.24	173.0	20.0

```
[21]: feature_corr = df[[
      "log_first_order_value",
      "log_first_order_items",
      "log_first_order_products"]].corr()
      feature_corr
```

```
[21]:
```

	log_first_order_value	log_first_order_items	\
log_first_order_value	1.000000	0.804424	
log_first_order_items	0.804424	1.000000	
log_first_order_products	0.560519	0.585234	

	log_first_order_products
log_first_order_value	0.560519
log_first_order_items	0.585234
log_first_order_products	1.000000

```
[22]: df[["log_first_order_value",
          "log_first_order_items",
          "log_first_order_products",
          "repeat_90d"]
      ].corr()["repeat_90d"].sort_values(ascending=False)
```

```
[22]: repeat_90d          1.000000
      log_first_order_value  0.143968
      log_first_order_items  0.120820
      log_first_order_products 0.107638
      Name: repeat_90d, dtype: float64
```

```
[23]: import statsmodels.api as sm
      from statsmodels.stats.outliers_influence import variance_inflation_factor
      X_vif = df[["log_first_order_value",
                  "log_first_order_items",
                  "log_first_order_products"]]

      X_vif = sm.add_constant(X_vif)
      vif = pd.DataFrame({"feature": X_vif.columns,
                          "VIF": [variance_inflation_factor(X_vif.values, i)
                                   for i in range(X_vif.shape[1])])})
      vif
```

```
[23]:
```

	feature	VIF
0	const	45.756344
1	log_first_order_value	2.935539
2	log_first_order_items	3.061967
3	log_first_order_products	1.575598

```
[24]: from sklearn.model_selection import train_test_split

      X = df[["log_first_order_value",
              "log_first_order_items",
              "log_first_order_products"]]
      y = df["repeat_90d"]
      X_train, X_test, y_train, y_test = train_test_split(
          X,
          y,
          test_size=0.20,
          random_state=42,
          stratify=y)

      print("Training set:", X_train.shape)
      print("Test set:", X_test.shape)
```

```
Training set: (4224, 3)
Test set: (1057, 3)
```

```
[25]: print("Overall repeat rate:", y.mean())
      print("Training repeat rate:", y_train.mean())
      print("Test repeat rate:", y_test.mean())
```

```
Overall repeat rate: 0.4726377580003787
```

Training repeat rate: 0.4725378787878788
Test repeat rate: 0.47303689687795647

```
[26]: baseline_accuracy = 1 - y_test.mean()  
print("Baseline accuracy:", round(baseline_accuracy, 4))
```

Baseline accuracy: 0.527

```
[28]: from sklearn.linear_model import LogisticRegression  
  
model = LogisticRegression(  
    max_iter=1000  
)  
  
model.fit(X_train, y_train)
```

```
[28]: LogisticRegression(max_iter=1000)
```

```
[30]: y_pred = model.predict(X_test)  
y_prob = model.predict_proba(X_test)[:, 1]
```

```
[31]: from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    roc_auc_score)  
  
print("Baseline Accuracy:", round(baseline_accuracy, 4))  
print("Model Accuracy:", round(accuracy_score(y_test, y_pred), 4))  
print("Precision:", round(precision_score(y_test, y_pred), 4))  
print("Recall:", round(recall_score(y_test, y_pred), 4))  
print("F1 Score:", round(f1_score(y_test, y_pred), 4))  
print("ROC-AUC:", round(roc_auc_score(y_test, y_prob), 4))
```

Baseline Accuracy: 0.527
Model Accuracy: 0.5402
Precision: 0.5207
Recall: 0.352
F1 Score: 0.42
ROC-AUC: 0.5704

```
[32]: #Model 1 Behavioral baseline  
coefficients = pd.DataFrame({  
    "feature": X.columns,  
    "coefficient": model.coef_[0],  
    "odds_ratio": np.exp(model.coef_[0])})  
coefficients.sort_values(  
    by="odds_ratio",  
    ascending=False)
```

```
"odds_ratio",
ascending=False)
```

```
[32]:
```

	feature	coefficient	odds_ratio
0	log_first_order_value	0.283876	1.328268
2	log_first_order_products	0.086947	1.090839
1	log_first_order_items	0.031032	1.031519

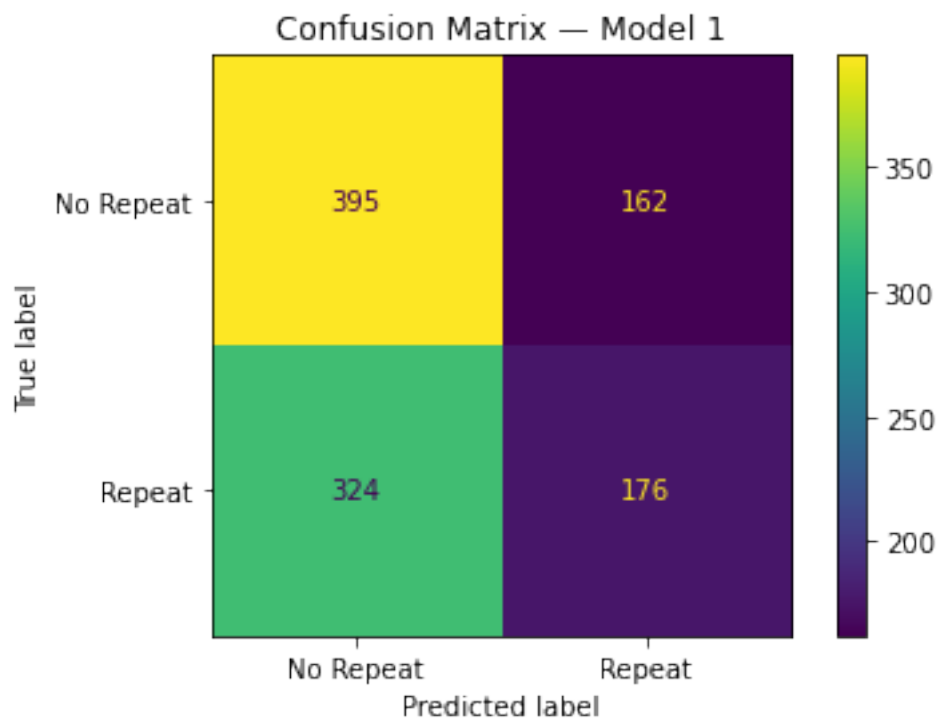
```
[ ]: ### Model 1: First-Order Behavioral Signals
```

The baseline logistic regression uses first-order value, item quantity, and
→ product variety to predict whether a customer makes a second purchase within
→ 90 days.

All three behavioral features show positive coefficients. First-order value has
→ the strongest positive association with repeat-purchase odds after
→ controlling for item quantity and product variety.

However, overall predictive performance remains limited (ROC-AUC = 0.570),
→ suggesting that first-order purchasing behavior alone does not sufficiently
→ distinguish repeat from non-repeat customers.

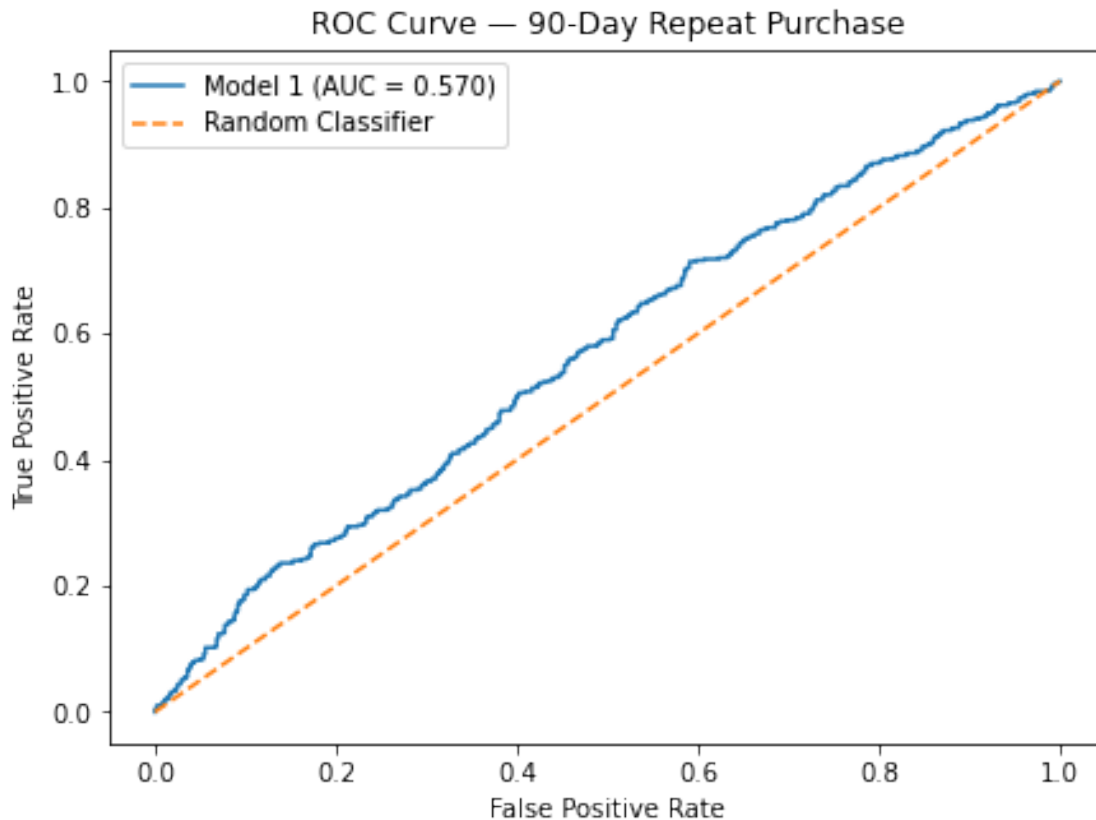
```
[33]: #confusion matrix
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Repeat", "Repeat"])
disp.plot()
plt.title("Confusion Matrix - Model 1")
plt.show()
```



```
[36]: #ROC Curve
from sklearn.metrics import roc_curve, roc_auc_score
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)

plt.figure(figsize=(7, 5))
plt.plot(fpr, tpr, label=f"Model 1 (AUC = {auc:.3f})")
plt.plot([0, 1],[0, 1],
         linestyle="--",
         label="Random Classifier")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - 90-Day Repeat Purchase")
plt.legend()

plt.show()
```



```
[ ]: ### Model 1 Evaluation
```

The behavioral model achieved a ROC-AUC of 0.570, compared with a random-classification benchmark of 0.500. Accuracy reached 54.0%, only modestly above the 52.7% majority-class baseline.

Recall was 35.2%, indicating that the model failed to identify a substantial share of customers who ultimately repeated within 90 days.

These results suggest that first-order spending, quantity, and product variety contain measurable retention signals, but are insufficient on their own for strong customer-level prediction.

At the default 0.50 classification threshold, the model correctly identified 176 of 500 repeat customers in the test set, while 324 repeat customers were classified as non-repeaters. This low recall highlights the limitations of using first-order purchase intensity alone for retention prediction.

```
[38]: # model 2
df["first_purchase_date"] = pd.to_datetime(
    df["first_purchase_date"])
df["acquisition_year"] = df["first_purchase_date"].dt.year
```

```
df["acquisition_month_num"] = df["first_purchase_date"].dt.month
print("Number of countries:", df["first_order_country"].nunique())

df["first_order_country"].value_counts().head(15)
```

Number of countries: 41

```
[38]: United Kingdom      4813
      Germany            93
      France             76
      Spain              34
      Belgium            26
      Netherlands        22
      Portugal           20
      Switzerland        19
      Sweden             19
      Australia          14
      Italy              14
      Channel Islands     13
      Austria            12
      Finland            11
      Norway             11
      Name: first_order_country, dtype: int64
```

```
[40]: # Create acquisition timing features
df["first_purchase_date"] = pd.to_datetime(
    df["first_purchase_date"])

df["acquisition_year"] = df["first_purchase_date"].dt.year
df["acquisition_month_num"] = df["first_purchase_date"].dt.month

# Create UK vs. international indicator
df["is_uk"] = (df["first_order_country"] == "United Kingdom")
               .astype(int)
df[["acquisition_year",
    "acquisition_month_num",
    "is_uk"]].head()
```

```
[40]:   acquisition_year  acquisition_month_num  is_uk
0           2009             12      1
1           2010             10      0
2           2010              9      0
3           2010              4      0
4           2011              2      0
```

```
[42]: df["months_since_start"] = (
    (df["first_purchase_date"].dt.year - 2009) * 12
```

```

    + (df["first_purchase_date"].dt.month - 12))
df[["first_purchase_date",
    "months_since_start"]].head()

```

```

[42]:   first_purchase_date  months_since_start
0  2009-12-14 08:34:00                0
1  2010-10-31 14:20:00               10
2  2010-09-27 14:59:00                9
3  2010-04-29 13:20:00                4
4  2011-02-02 16:01:00               14

```

```

[47]: features_model2 = [
        "log_first_order_value",
        "log_first_order_items",
        "log_first_order_products",
        "months_since_start",
        "is_uk"]
X2 = df[features_model2]
y = df["repeat_90d"]

X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2,y,test_size=0.20, random_state=42,stratify=y)
model2 = LogisticRegression(max_iter=1000)
model2.fit(X2_train, y2_train)

y2_pred = model2.predict(X2_test)
y2_prob = model2.predict_proba(X2_test)[: , 1]

```

```

[49]: model_comparison = pd.DataFrame({
        "Metric": ["Accuracy","Precision","Recall","F1 Score","ROC-AUC"],
        "Model 1": [
            accuracy_score(y_test, y_pred),
            precision_score(y_test, y_pred),
            recall_score(y_test, y_pred),
            f1_score(y_test, y_pred),
            roc_auc_score(y_test, y_prob)],
        "Model 2": [
            accuracy_score(y2_test, y2_pred),
            precision_score(y2_test, y2_pred),
            recall_score(y2_test, y2_pred),
            f1_score(y2_test, y2_pred),
            roc_auc_score(y2_test, y2_prob)]})

model_comparison.round(3)

```

```

[49]:   Metric  Model 1  Model 2
0  Accuracy    0.540    0.571

```

1	Precision	0.521	0.554
2	Recall	0.352	0.482
3	F1 Score	0.420	0.516
4	ROC-AUC	0.570	0.585

```
[50]: model2_coefficients = pd.DataFrame({
    "feature": X2.columns,
    "coefficient": model2.coef_[0],
    "odds_ratio": np.exp(model2.coef_[0])})
model2_coefficients.sort_values(
    "coefficient",
    ascending=False)
```

	feature	coefficient	odds_ratio
0	log_first_order_value	0.260785	1.297949
2	log_first_order_products	0.099683	1.104821
1	log_first_order_items	0.038910	1.039677
3	months_since_start	-0.032338	0.968180
4	is_uk	-0.056005	0.945534

[]: *### Model 2: Adding Acquisition Context*

Model 2 extends the behavioral model by incorporating acquisition timing and whether the customer's first purchase occurred in the UK market. Adding acquisition context improved performance across all evaluated metrics. Accuracy increased from 54.0% to 57.1%, while recall increased from 35.2% to 48.2%. ROC-AUC improved more modestly, from 0.570 to 0.585. The results indicate that acquisition context contributes incremental information beyond first-order purchasing behavior, particularly in identifying a larger share of customers who subsequently repeat. However, overall discrimination remains limited, suggesting that first-purchase information alone is insufficient for strong customer-level retention prediction.

```
[51]: # ablation test
from sklearn.metrics import roc_auc_score

feature_sets = {
    "Behavior Only": [
        "log_first_order_value",
        "log_first_order_items",
        "log_first_order_products"],
    "Behavior + Time": [
        "log_first_order_value",
        "log_first_order_items",
        "log_first_order_products",
        "months_since_start"],
    "Behavior + Country": [
```

```

        "log_first_order_value",
        "log_first_order_items",
        "log_first_order_products",
        "is_uk"],
    "Behavior + Time + Country": [
        "log_first_order_value",
        "log_first_order_items",
        "log_first_order_products",
        "months_since_start",
        "is_uk"]}]
results = []
for name, features in feature_sets.items():
    X_temp = df[features]
    X_train_temp, X_test_temp, y_train_temp, y_test_temp = train_test_split(
        X_temp, y, test_size=0.20,
        random_state=42, stratify=y)
    temp_model = LogisticRegression(max_iter=1000)
    temp_model.fit(X_train_temp, y_train_temp)
    y_prob_temp = temp_model.predict_proba(X_test_temp)[: , 1]
    auc_temp = roc_auc_score(y_test_temp, y_prob_temp)
    results.append({"Model": name, "ROC_AUC": auc_temp})

ablation_results = pd.DataFrame(results)
ablation_results

```

```

[51]:
      Model  ROC_AUC
0  Behavior Only  0.570449
1  Behavior + Time  0.586689
2  Behavior + Country  0.570418
3  Behavior + Time + Country  0.585206

```

[]: *### Feature Ablation: Acquisition Context*

To identify the source of Model 2's incremental predictive performance,
 ↳ acquisition timing and geography were evaluated separately.
 Adding acquisition timing increased ROC-AUC from 0.570 to 0.587, while adding
 ↳ country alone produced virtually no improvement (0.570). Combining both
 ↳ features resulted in an ROC-AUC of 0.585, slightly below the time-only
 ↳ specification.
 This indicates that the improvement in predictive performance is primarily
 ↳ associated with acquisition timing rather than geography. Country was
 ↳ therefore excluded from the more parsimonious final specification.

```

[53]: cohort_retention = (
    df.groupby("acquisition_month")
      .agg(new_customers=("customer_id", "count"),
          repeat_customers_90d=("repeat_90d", "sum"),
          repeat_rate_90d=("repeat_90d", "mean")).reset_index()

```

```
cohort_retention["repeat_rate_90d"] *= 100
cohort_retention
```

```
[53]:
```

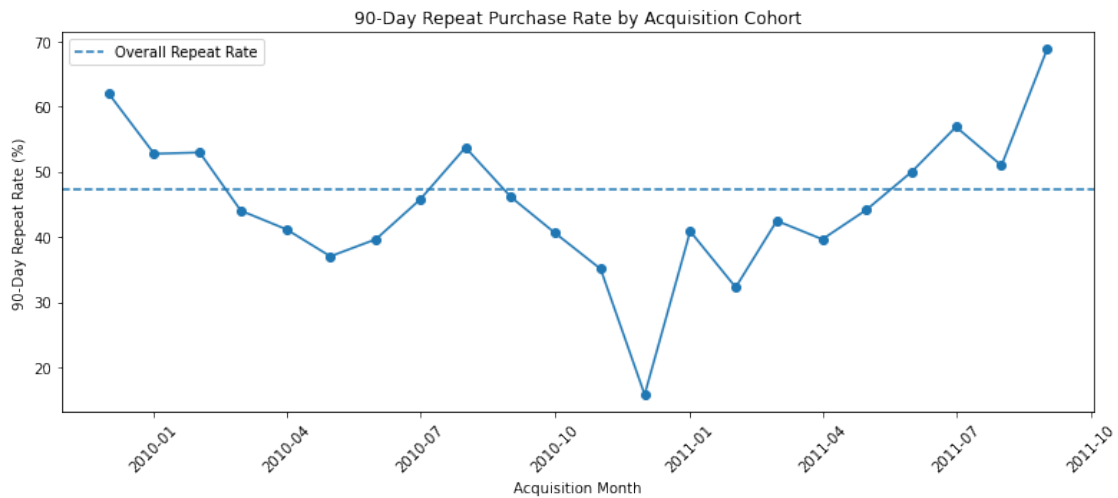
	acquisition_month	new_customers	repeat_customers_90d	repeat_rate_90d
0	2009-12	955	592	61.989529
1	2010-01	383	202	52.741514
2	2010-02	374	198	52.941176
3	2010-03	443	195	44.018059
4	2010-04	294	121	41.156463
5	2010-05	254	94	37.007874
6	2010-06	270	107	39.629630
7	2010-07	186	85	45.698925
8	2010-08	162	87	53.703704
9	2010-09	243	112	46.090535
10	2010-10	377	153	40.583554
11	2010-11	325	114	35.076923
12	2010-12	76	12	15.789474
13	2011-01	71	29	40.845070
14	2011-02	124	40	32.258065
15	2011-03	179	76	42.458101
16	2011-04	106	42	39.622642
17	2011-05	111	49	44.144144
18	2011-06	108	54	50.000000
19	2011-07	102	58	56.862745
20	2011-08	106	54	50.943396
21	2011-09	32	22	68.750000

```
[56]: cohort_retention["acquisition_month"] = pd.to_datetime(
        cohort_retention["acquisition_month"])

plt.figure(figsize=(11, 5))
plt.plot(
    cohort_retention["acquisition_month"],
    cohort_retention["repeat_rate_90d"],
    marker="o")
plt.axhline(
    y=df["repeat_90d"].mean() * 100,
    linestyle="--",
    label="Overall Repeat Rate")

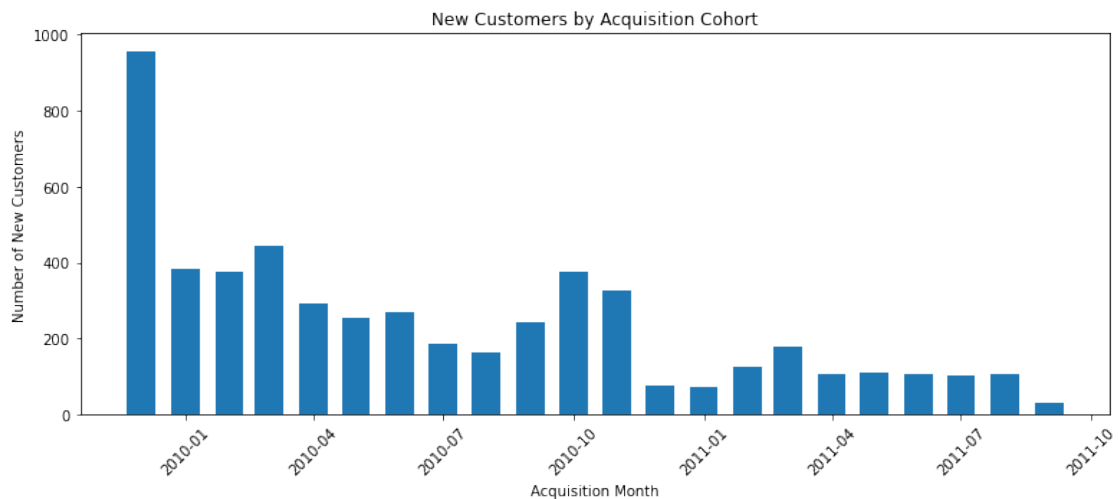
plt.title("90-Day Repeat Purchase Rate by Acquisition Cohort")
plt.xlabel("Acquisition Month")
plt.ylabel("90-Day Repeat Rate (%)")
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
```

```
plt.show()
```



```
[57]: plt.figure(figsize=(11, 5))
plt.bar(
    cohort_retention["acquisition_month"],
    cohort_retention["new_customers"],
    width=20)
plt.title("New Customers by Acquisition Cohort")
plt.xlabel("Acquisition Month")
plt.ylabel("Number of New Customers")
plt.xticks(rotation=45)
plt.tight_layout()

plt.show()
```



```
[ ]: ### Acquisition Cohort Patterns
90-day repeat purchase rates vary substantially across acquisition cohorts and
↳do not follow a simple linear time trend. Retention declined across several
↳early cohorts, reached a pronounced low around late 2010, and subsequently
↳increased across later cohorts.
However, cohort sizes also declined substantially over the observation period.
↳The most extreme repeat rates therefore occur in some of the smaller cohorts
↳and should be interpreted cautiously.
These patterns suggest that acquisition timing may capture nonlinear cohort
↳effects rather than a simple month-to-month trend. A categorical cohort
↳specification is therefore evaluated as an alternative to the linear time
↳feature.
```

```
[58]: df["acquisition_quarter"] = (df["first_purchase_date"]
    .dt.to_period("Q")
    .astype(str))

df["acquisition_quarter"].value_counts().sort_index()
```

```
[58]: 2009Q4      955
      2010Q1     1200
      2010Q2      818
      2010Q3      591
      2010Q4      778
      2011Q1      374
      2011Q2      325
      2011Q3      240
      Name: acquisition_quarter, dtype: int64
```

```
[59]: #model 3: behavior+acquisition quarter
quarter_dummies = pd.get_dummies(
    df["acquisition_quarter"],
    prefix="quarter",
    drop_first=True,
    dtype=int)

quarter_dummies.head()
```

```
[59]:   quarter_2010Q1  quarter_2010Q2  quarter_2010Q3  quarter_2010Q4  \
0                0                0                0                0
1                0                0                0                1
2                0                0                1                0
3                0                1                0                0
4                0                0                0                0
```

	quarter_2011Q1	quarter_2011Q2	quarter_2011Q3
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	1	0	0

```
[60]: behavior_features = df[
        ["log_first_order_value",
         "log_first_order_items",
         "log_first_order_products"]]
X3 = pd.concat([behavior_features,
                quarter_dummies],axis=1)

X3.head()
```

```
[60]:   log_first_order_value  log_first_order_items  log_first_order_products  \
0                3.828641                2.397895                0.693147
1                6.417598                6.234411                3.713572
2                5.407889                5.924256                3.044522
3                6.974965                6.161207                3.850148
4                5.815324                5.288267                2.890372
```

	quarter_2010Q1	quarter_2010Q2	quarter_2010Q3	quarter_2010Q4	\
0	0	0	0	0	
1	0	0	0	1	
2	0	0	1	0	
3	0	1	0	0	
4	0	0	0	0	

	quarter_2011Q1	quarter_2011Q2	quarter_2011Q3
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	1	0	0

```
[63]: X3_train, X3_test, y3_train, y3_test = train_test_split(
        X3,y,test_size=0.20,random_state=42,stratify=y)
model3 = LogisticRegression(max_iter=1000)
model3.fit(X3_train, y3_train)
y3_pred = model3.predict(X3_test)
y3_prob = model3.predict_proba(X3_test)[:, 1]
```

```
[66]: final_model_comparison = pd.DataFrame({
        "Metric": [
            "Accuracy",
```

```

        "Precision",
        "Recall",
        "F1 Score",
        "ROC-AUC"],
    "Behavior Only": [
        accuracy_score(y_test, y_pred),
        precision_score(y_test, y_pred),
        recall_score(y_test, y_pred),
        f1_score(y_test, y_pred),
        roc_auc_score(y_test, y_prob)],
    "Behavior + Linear Time": [
        accuracy_score(y2_test, y2_pred),
        precision_score(y2_test, y2_pred),
        recall_score(y2_test, y2_pred),
        f1_score(y2_test, y2_pred),
        roc_auc_score(y2_test, y2_prob)],
    "Behavior + Quarter": [
        accuracy_score(y3_test, y3_pred),
        precision_score(y3_test, y3_pred),
        recall_score(y3_test, y3_pred),
        f1_score(y3_test, y3_pred),
        roc_auc_score(y3_test, y3_prob)]]})

final_model_comparison.round(3)

```

[66]:	Metric	Behavior Only	Behavior + Linear Time	Behavior + Quarter
0	Accuracy	0.540	0.571	0.593
1	Precision	0.521	0.554	0.585
2	Recall	0.352	0.482	0.484
3	F1 Score	0.420	0.516	0.530
4	ROC-AUC	0.570	0.585	0.617

[]: *### Model 3: Capturing Nonlinear Acquisition Cohort Effects*

Exploratory cohort analysis showed that 90-day repeat rates did **not** follow a
 ↳ simple linear trend over time. To capture these nonlinear differences, ↳
 ↳ acquisition timing was re-specified **as** quarterly cohort categories rather ↳
 ↳ than a continuous month index.

The quarter-based model improved ROC-AUC **from** 0.585 to 0.617 **and** accuracy **from** ↳
 ↳ 57.1% to 59.3%. Precision increased to 58.5%, **while** recall remained ↳
 ↳ approximately 48%.

This improvement suggests that retention differences are better represented by ↳
 ↳ cohort-level variation than by a simple linear time trend. Acquisition ↳
 ↳ context therefore provides useful incremental predictive information beyond ↳
 ↳ first-order purchase behavior.

```

[67]: model3_coefficients = pd.DataFrame({
        "feature": X3.columns,

```

```

        "coefficient": model3.coef_[0],
        "odds_ratio": np.exp(model3.coef_[0]))

model3_coefficients.sort_values(
    "coefficient",
    ascending=False)

```

```

[67]:

```

	feature	coefficient	odds_ratio
0	log_first_order_value	0.255108	1.290601
2	log_first_order_products	0.113308	1.119977
1	log_first_order_items	0.042567	1.043486
9	quarter_2011Q3	-0.293889	0.745360
3	quarter_2010Q1	-0.552920	0.575268
5	quarter_2010Q3	-0.577744	0.561163
8	quarter_2011Q2	-0.703505	0.494848
7	quarter_2011Q1	-0.972125	0.378278
4	quarter_2010Q2	-0.985294	0.373330
6	quarter_2010Q4	-1.091726	0.335637

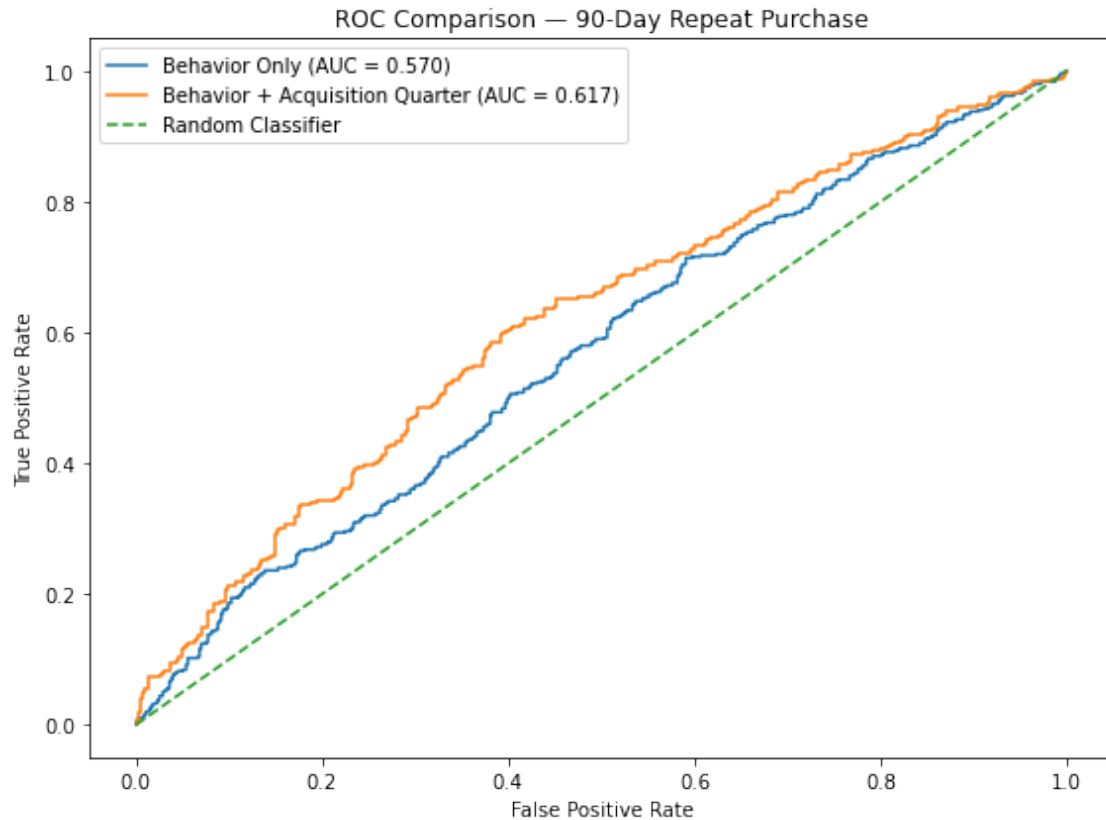
```

[68]: from sklearn.metrics import roc_curve, roc_auc_score

# Model 1
fpr1, tpr1, _ = roc_curve(y_test, y_prob)
auc1 = roc_auc_score(y_test, y_prob)
# Model 3
fpr3, tpr3, _ = roc_curve(y3_test, y3_prob)
auc3 = roc_auc_score(y3_test, y3_prob)
plt.figure(figsize=(8, 6))
plt.plot(
    fpr1, tpr1,
    label=f"Behavior Only (AUC = {auc1:.3f})")
plt.plot(
    fpr3, tpr3,
    label=f"Behavior + Acquisition Quarter (AUC = {auc3:.3f})")
plt.plot(
    [0, 1], [0, 1], linestyle="--", label="Random Classifier")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Comparison - 90-Day Repeat Purchase")
plt.legend()
plt.tight_layout()
plt.show()

```



[]: *Final Model Interpretation*

The final logistic regression model combines first-order purchase behavior with quarterly acquisition cohorts.

First-order value remained the strongest positive behavioral signal after controlling for acquisition timing, while product variety and item quantity also showed positive associations with 90-day repeat purchasing.

Incorporating acquisition cohorts improved ROC-AUC from 0.570 to 0.617 and increased accuracy from 54.0% to 59.3%. The improvement supports the earlier cohort analysis, which showed substantial nonlinear variation in retention across acquisition periods.

Despite the improvement, predictive performance remains moderate. First-purchase behavior and acquisition context provide useful retention signals, but they are not sufficient for highly accurate customer-level prediction.

[]: *Business Insights & Recommendations*

Key Findings

1. The first 90 days represent a critical retention window

Across eligible customers, 23.4% made a repeat purchase within 30 days,
→increasing to 38.3% within 60 days and 47.3% within 90 days.

This suggests that a substantial share of repeat behavior develops during the
→first three months after acquisition, making the post-purchase period an
→important window for retention efforts.

2. Higher-value first orders are associated with stronger retention

90-day repeat rates increased consistently across first-order value quartiles:

- Q1: 36.6%
- Q2: 45.2%
- Q3: 50.2%
- Q4: 57.1%

Customers with larger initial purchases therefore showed substantially stronger
→subsequent retention. First-order value also remained the strongest positive
→behavioral signal in the logistic regression models after controlling for
→other first-order characteristics.

3. Broader product exploration is also associated with repeat purchasing

Customers purchasing a wider variety of products on their first order showed
→higher 90-day repeat rates:

- Q1: 41.9%
- Q2: 44.1%
- Q3: 48.5%
- Q4: 54.6%

This pattern suggests that initial product breadth may serve as an additional
→indicator of customer engagement.

4. Acquisition cohort matters

Retention varied substantially across acquisition periods and did not follow a
→simple linear time trend.

Representing acquisition timing as quarterly cohorts improved model ROC-AUC
→from 0.570 to 0.617, outperforming both the behavior-only model and the
→linear-time specification.

Geography, by comparison, provided virtually no incremental predictive value
→when tested as UK versus international customers.

5. First-purchase data identifies retention signals, but does not fully →predict individual behavior

The final model improved accuracy from 54.0% to 59.3% and ROC-AUC from 0.570 to
→0.617 after incorporating acquisition cohort information.

However, predictive performance remained moderate. This indicates that
→first-order behavior is useful for customer segmentation and early retention
→targeting, but additional behavioral and engagement data would be needed for
→stronger individual-level prediction.

Business Recommendations

```

**Prioritize the first 90 days after acquisition.**
Develop structured post-purchase retention journeys during this period,
    ↳including product education, replenishment or complementary-product
    ↳recommendations, and appropriately timed re-engagement campaigns.
**Use first-order behavior for early customer segmentation.**
First-order value and product breadth can help identify customers showing
    ↳stronger initial engagement. Rather than treating these variables as
    ↳deterministic predictions, use them as signals for differentiated retention
    ↳strategies.
**Investigate acquisition cohorts before evaluating retention performance.**
Because repeat rates vary substantially across acquisition periods, retention
    ↳KPIs should be monitored by cohort rather than only through aggregate
    ↳averages. Differences in campaign mix, promotions, seasonality, or
    ↳acquisition channels may help explain these patterns if additional data
    ↳becomes available.
**Expand future retention models with post-acquisition signals.**
Future modeling could incorporate website engagement, email interaction,
    ↳promotion exposure, acquisition channel, product category, and customer
    ↳activity after the first purchase. These variables may capture behavioral
    ↳intent that cannot be observed from the first transaction alone.

```

```
[ ]: # Customer Repeat Purchase & Retention Analysis
```

```

## Executive Summary
This project analyzes more than one million retail transaction records to
    ↳understand the drivers of 90-day customer repeat purchasing.
Using SQL, customer-level purchase histories were constructed from raw
    ↳transaction data, followed by cohort and behavioral analysis in Python.
Key findings include:
- 47.3% of eligible customers made a repeat purchase within 90 days.
- 90-day repeat rates increased from 36.6% to 57.1% across first-order value
    ↳quartiles.
- Customers purchasing a broader variety of products on their first order also
    ↳showed higher retention.
- Acquisition cohorts exhibited substantial differences in repeat behavior.
- Adding quarterly acquisition cohorts improved logistic regression ROC-AUC
    ↳from 0.570 to 0.617.
- First-purchase behavior provides useful early retention signals, but remains
    ↳insufficient for highly accurate individual-level prediction.

The analysis suggests that businesses can use first-order behavior for early
    ↳customer segmentation while focusing retention efforts on the first 90 days
    ↳and monitoring performance through acquisition cohorts.

```